Elio Rocha and Nicholas Dalsass 1

Dr. Minhee Jun

CSC 370

11 / 26 / 24

Compiler Project Report

Introduction & Objectives of the Project:

Understanding the complexities of a compiler is essential in truly grasping how source code written to a machine operates "under the hood." A compiler is vital in turning this source code into executable machine code for the CPU to execute and run. The most popular languages such as C++, Java, and Python all use some type of compiler to construct their language into executable code that the machine can understand. In this project, the objective is to not only create a compiler, but to understand each phase of the compiler and to construct our own small-scale programming language.

Design & Implementation Details:

Syntax: Before diving into compiler design, let's explain our language's syntax
Lexical Analyzer: Turns source code into tokens
Syntax and Semantic Analyzer: Checks for consistent logic using our grammar and sorts tokens into parse trees according to our grammar rules
Intermediate Code Generation: Generates bytecode
Code Generation: Virtual Machine executes bytecode

To have a more general understanding of how a compiler works, we must first understand the basic phases of a compiler. These include the lexical analyzer, syntax analyzer, semantic analyzer, intermediate code generation, and code generation. The first phase, the lexical analyzer, is sometimes known as the tokenizer. This tokenizer turns a given chunk of source code into tokens, represented by the token name and its lexeme value. Some examples of token types in this project include: EQUAL_OP, INT_LIT, IF and IDENTIFIER. Using Java's record class, it was fairly easy to implement our desired tokens into the project. Here are a few examples…

```
public record Token(TokenType tokenType, String value) {
   public enum TokenType {
        PLUS_OP,
        IDENTIFIER,
        MULTI_OP,
        INT_LIT,
        EQUAL_OP
}
```

Figure 1: A Java implementation of a Token record

The syntax and the tokenizer go hand in hand since the tokens essentially account for all of the necessary operators, characters, keywords, variables, statements, etc. which we will incorporate into our compiler. For our purposes, our syntax consists of the following: basic print statements, conditionals, while loops, arithmetic operations, simple variable assignments, and a few other minor bits of functionality. We chose a simple route, using a general variable

declaration called ‘var’ as our only way to declare and store an integer variable. To start the program, we used a keyword called ‘start’ which we tokenized to denote the start of a program, followed by a set of parentheses and brackets to resemble something similar to a Java method.

Tokenizer:

To begin the tokenizer process, we first create a string named contents that reads all the content (source code) from a file into a string. Then, we create an integer index to keep track of our position in the file. We begin tokenizing contents using the tokens() method which returns an array list of the tokens. The tokenizer will read character by character, entering one of many branched if statements which align with its corresponding condition. For example, if the character is a letter at the specified position in contents, it will enter an if statement where it will begin to build a string continuously by moving through contents until the condition is no longer satisfied. It will then compare the built string to related token types until one is matched. Once the built string or lexeme is matched with a token type, it will be added to the array list of tokens and the string builder will be cleared so that the next string can be built.

```
while (index < contents.length()) {

   if (Character.isLetter(contents.charAt(index))) {
       buf.append(c());
       while (Character.isLetter(contents.charAt(index)) ||
Character.isDigit(contents.charAt(index))) {
           buf.append(c());
       }
       if (buf.toString().equals("start")) {
           tokens.add(new Token(Token.TokenType.START, buf.toString()));
           clearBuf();
       } else if (buf.toString().equals("print")) {
           tokens.add(new Token(Token.TokenType.PRINT, buf.toString()));
           clearBuf();
       }
```

Figure 2: A Java implementation of the Tokenizer class’s operation on the source code

It is important to note that if the tokenizer comes across a whitespace, it will ignore that character. Likewise, if an unknown character is introduced into the tokenizer, an error message will be prompted to the user via the terminal. The tokenizer will continue to tokenize the source code program until the index is equal to the content’s size. Once the tokenizer has successfully tokenized the source code’s contents, the array list of tokens will be sent to the parser and the next step in compilation will begin.

Syntax Analyzer:

Now that we have established the basic syntax of our language and have successfully tokenized all of our source code into its basic components, we must assign a grammar to our language which analyzes our tokenized code and performs the operations which we have assigned to it. We chose to represent our grammar in the form of an EBNF, for simplicity. Here is what it looks like:

```
<program> --> <start>
   <start> --> “start” “()” “{” <stmt>* “}”
   <stmt> --> <assign> | <print> | <if> | <while>
   <if> --> “if” “(” <condition> “)” “{” <stmt> “}” [“else” “{” <stmt>* “}”]
   <while> --> “while” “(” <condition> “)” “{” <stmt> “}”
   <assign> --> <var> IDENTIFIER “=” <expr> “;”
   <print> → “print” “(” (STRING_LIT | <expr>) “)” “;”
   <condition> --> <factor> {(< | > | >= | <= | == | !=) <factor>}
   <expr> --> <term> {(+|-) <term>}
   <term> --> <factor>{(*|/)<factor>}
   <var> --> VARIABLE
   <factor> --> INT_LIT | IDENTIFIER | “(” <expr> “)”
```

Figure 3: EBNF representation of a simple grammar with prescribed functionality

Parsing our Grammar:

Great, we've established a grammar for our language… Now what? The next step in our syntactic analysis is to parse the grammar according to the rules which we have prescribed in our grammar. Here is a tree which demonstrates how we parsed the source code according to our grammar:

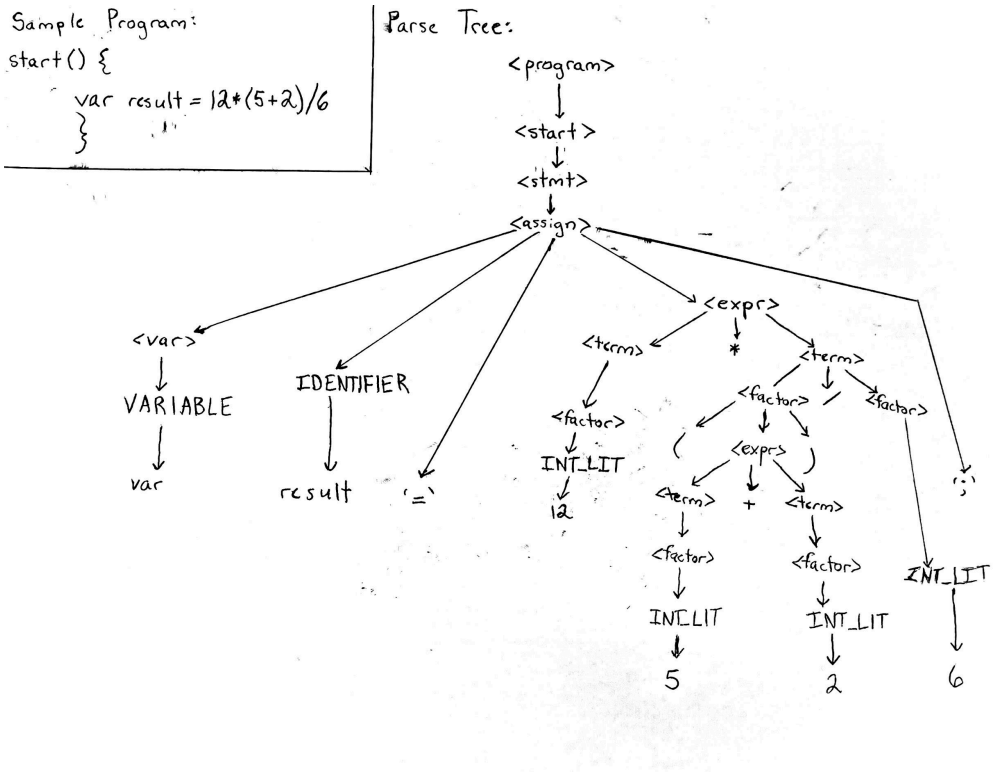

Figure 4: An arithmetic sample program parsed into a tree according to our grammar rules

For simplicity's sake, we elected to use the recursive-descent parsing method. This is a method of top-down parsing which makes a method for each non-terminal in our grammar. Here is a snippet of how a recursive descent parser calls itself in order to align our source code with the prescribed grammar:

```
  public void stmt() {
         do{
             while (peekToken(0) == Token.TokenType.VARIABLE &&    peekToken(1) ==
 Token.TokenType.IDENTIFIER) {
                 assign();
             }
             while (peekToken(0) == Token.TokenType.PRINT) {
                 nextTokenIndex();
                 print();
             }
             while (peekToken(0) == Token.TokenType.IF) {
                 nextTokenIndex();
                 _if();
             }
             while (peekToken(0) == Token.TokenType.WHILE) {
                 nextTokenIndex();
                 _while();
             }

          } while (tokens.get(tokenIndex).tokenType() !=
 Token.TokenType.RIGHT_BRACK);
     }
```

Figure 5: A Java implementation of recursive descent parsing

The code in Figure 4 gives us a glimpse into what goes into a recursive top-down parse. We first create a counter for our token index beginning with zero and check what the token corresponds to in our Token class. According to the result, the token index will be incremented and the next method called. Control may return to the caller at any time or may be passed to another function, such as assign( ), factor( ), or expr( ) until the last token is parsed. A quick look at our code from our term( ) method can help to understand this parsing concept a bit better:

```
public void term() {
        factor();
        while (Token.TokenType.MULTI_OP == tokens.get(tokenIndex).tokenType()
                || Token.TokenType.DIV_OP == tokens.get(tokenIndex).tokenType()) {
            int i = tokenIndex;
            nextTokenIndex();
            factor();
}
```

Figure 6: Java method called term() to parse arithmetic expressions into terms and factors

Intermediate Code Generation:

This compiler project will produce *bytecode* instructions instead of the traditional method of compilation in which assembly language is generated and promptly translated into

machine code. This project will include our own virtual machine which will run the bytecode instructions once fully generated. That being said, we now enter the intermediate code generation phase in the compiler design process.

It is important to understand what bytecode is and why it is being implemented in place of assembly language. Bytecode is a lower level language which represents a set of instructions. In other words, it is a low-level intermediate form of code. The source code generates bytecodes and this bytecode is executed by a stack-based virtual machine in which each instruction from a stack is executed from top to bottom. Traditionally, languages like C++ transform the source code into assembly code and then run it by translation into machine code. We decided against this route and followed Java's approach of using bytecode and a virtual machine. Moreover, the implementation of bytecode and a virtual machine allows the language to be portable, meaning it can be executed on any type of platform unlike a language like C++ which depends on the user's platform architecture to generate and execute assembly.

After the semantic analysis phase of the compiler, bytecode generation is usually next in line. In a similar manner as our Token record implementation, we used a record to represent the bytecode implementation:

```
public record Bytecode(Opcode opcode, String operand) {
   public enum Opcode {
        CONST,  //loads into stack
        ADD,    //add
        SUB,    //sub
        MULTIPLY,   //multiply
        DIV,    //divide
        ILT,    //int less than
        IGT,    //int greater than
        ILTET,  //int less than or equal to
        IGTET,  //int greater than or equal to
        PRINT,  //print
        PRINTV, //print variable?
        GLOAD,  //load global variable
        GSTORE,    //store global variable
        BRT,    //branch if true
        BRF,    //branch if false
        HALT
   }
}
```

Figure 7: A Java implementation of the Bytecodes class used in this project

Since the number of bytecode instructions depends on the size and complexity of the source code, we used a flexible array list to add bytecode instructions throughout the execution of the parser. It is important to maintain the order of bytecode instructions loaded into the array list because of how the virtual machine will read and execute each instruction using a stack. Our bytecode instructions are generated in our parse tree where, after a key token is parsed, it creates the parse code.

```
public void factor() {
   if (Token.TokenType.INT_LIT == tokens.get(tokenIndex).tokenType()) {
        bytecodes.add(new Bytecode(Bytecode.Opcode.CONST,
```

```
(tokens.get(tokenIndex).value())));
        if (previousToken(1) == Token.TokenType.EQUAL_OP && previousToken(2) ==
Token.TokenType.IDENTIFIER
            && previousToken(3) == Token.TokenType.VARIABLE) {

            bytecodes.add(new Bytecode(Bytecode.Opcode.GSTORE, Integer.toString(j)));
        }
}
```

Figure 8: A Java implementation of our factor( ) method which generates bytecode

For example, in Figure 8, in our factor( ) method, rooted from the expr( ) method which focuses mainly on our arithmetic grammar, adds a bytecode instruction to the ArrayList *bytecodes*. It will create a bytecode with its opcode as CONST and the operand with the integer literal value. For example, if the user wanted to print the sum of 5+3, the bytecode instructions would be as follows: CONST 5, CONST 3, ADD, PRINT. The opcode CONST will load its operand to a stack which contains 5 and 3, then the opcode ADD will pop both numbers from the stack and push the sum back into the stack. Finally, the PRINT opcode will pop the result form the stack and print it out the following sum, 8.

Code Generation:

After all the bytecode instructions have been generated and added to the *bytecodes* array list, control will now be passed to the virtual machine for it to be read and executed. Given a program (source code) by the user, a list of bytecodes will be generated and read by the virtual machine. As stated before, we took inspiration from Java's Virtual Machine which is stack-based and constructed our own mini virtual machine using a similar stack-based approach. Each opcode of the bytecode will undergo an operation. We used a while loop to continuously process each index of opcode until the HALT opcode is read which will indicate the end of the program.

```
while (ip < bytecodes.size()) {
   Bytecode.Opcode opcode = bytecodes.get(ip).opcode();    //fetch for opcode
   ip++;
   switch(opcode) {
        case CONST:
            stack[++sp] = bytecodes.get(op++).operand();
            break;
        case ADD:
            b = stack[sp--];
            a = stack[sp--];
            c = Integer.parseInt(a) + Integer.parseInt(b);
            stack[++sp] = Integer.toString(c);
            op++;
            break;
}
```

Figure 9: A code snippet of our virtual machine

The *sp* variable represents the position in the stack which pushes and pops values onto the stack. The *ip* variable represents the instruction pointer indicating where the instruction index is pointing to to fetch the opcode. Similarly, the op variable represents the operand pointer to get the operand of the bytecode instruction. Below is a test example of our program that will be read and executed via our compiler.

```
start() {
   var x = 7;
   if (x < 5) {
       print("x is less than 5");
   }

   print(x);
}
```

Figure 10: An example of a test program of our language

The following program will be tokenized, parsed and generate the following bytecode instructions:

```
CONST 7
GSTORE 0
GLOAD 0
CONST 5
ILT
BRF 8
CONST “x is less than 5”
PRINT
GLOAD 0
PRINTV
HALT
```

Figure 11: Bytecode generated by the program in Figure 9

As seen in Figure 11, a total of 10 bytecode instructions were generated by the program demonstrated in Figure 9. The first instruction, CONST 7 will load 7 into the stack. The GSTORE 0 will pop 7 from the stack and store it into a global array called global that holds variables. Next, GLOAD 0 will push the 7 from the globals array back into the stack. The next instruction, CONST 5 will push 5 onto the stack.Then, the ILT instruction will pop the first 2 numbers from the stack and check if 7 is less than 5, it will push the boolean result onto the stack. Since the result of the comparison is false, false will be pushed into the stack. The BRF 8 instruction will check the top of the stack to see whether the last statement was false or true; since it is false, the operand 8 will now point to instruction 8, branching across the instructions CONST “x is less than 5” and PRINT. Now that the program branched to the instruction, PRINTV, the instruction now prints the global variable stored in index 0 of the globals array. Lastly, the instruction HALT will stop the execution indicating that the program is fully completed. The program in Figure 10 highlights a simple program that will print “x is less than 5” if the condition of the if statement is true. This demonstrates how we implemented *if* statements into our compiler and utilized the bytecode opcode BRF to branch ahead and skip the if statement block if the condition is not satisfied.

```
start() {
   var x = 0;
   while (x < 10) {
       x = x + 1;
```

```
        print(x);
    }
}
```

Figure 12: An example of a while loop in our program

In Figure 12, our program first initializes a variable x to 0, then increments and prints x if the condition of the while loop is true until x is no longer less than 10. One of the key aspects in implementing the while loop was utilizing the opcode BR. The opcode BR with an operand of 2 is placed at the end of the loop body. The BR instruction would branch back to the while loop's condition (x < 10) to check if x still satisfies the condition. The BRF branch would branch past the loop body when x is no longer less than 10. In other words, it would skip past the while loop if the condition is not met. This BR opcode mechanism allowed us to effectively implement a simple while loop in our language.

Challenges Encountered & Solutions:

Once we had completed our lexical analyzer, the first big step in the project, we hit a roadblock concerning the actual method of parsing. We had learned different parsing methods in class but were unsure of how to actually apply them. Our solution was to construct a grammar which performed proper precedence arithmetic and had the correct syntax for our statements. Top-down, recursive descent parsing was the key here and allowed us to solve the parsing problem using the grammar we had created.

Now that we had parsed our grammar according to the rules we constructed, we were not sure how we would actually execute the code. Our solution was to create a bytecode opcode array which performed low-level operations in order, as orchestrated by the Virtual Machine. This was the key to breaking down the actual execution of our code.

Conclusion & Future Improvements:

Our current compiler provides basic implementations of top-down parsing methodology and bytecode generation. There are, however, still several areas where future improvements could be made which could significantly improve the performance and logic of the compiler. For instance, an AST would be more ideal and far more efficient in generating our bytecode since we currently generate the bytecode using the parse tree. This could significantly increase the readability of our code and allow for manipulation and easier control of the program's flow. In addition, we could expand on the language and apply several more key programming language concepts such as switch statements, do-while loops, and more. By expanding on our language and the compiler itself, we would be enhancing the functionality of our compiler and its usage.

Overall, a great deal can be learned from building a small-scale compiler. This process unlocks what is actually going on behind the scenes when you write a line of code. Though Java was our language of choice, simpler languages like C have a more direct correlation with what actions the CPU is performing on account of your code. We chose to write the code for our compiler in Java since it is a C-based language and we were generally more familiar with its syntax. All in all, we learned a great deal about the inner workings of a programming language and what goes on behind the scenes when creating a compiler.